
REQUIREMENT ANALYSIS DOCUMENT (RAD): TREND WAVE (ONLINE SHOPPING MANAGEMENT SYSTEM)

Name: B Lakshanyawanti
Course: B.Sc Artificial Intelligence

Subject: Database Management System
Year: 2nd year

1. EXECUTIVE PROJECT SUMMARY

1.1 Purpose of the Online Shopping Management System

The TrendWave is digital commercial network that allows remote consumers to search for products, view real-time inventory counts, allocate specific items to a virtual shopping basket, process electronic payments, and track courier dispatches. Processing immediate stock checkouts accurately requires a robust, real-time transaction workflow: when a customer adds the final remaining unit of a product to their order, that specific inventory item must be locked instantly to prevent duplicate over-selling, financial card validation must clear, and a digital invoice reference must be dispatched automatically.

1.2 Why a Relational Database Management System (RDBMS) is Required

To manage this safely, the system relies on a Relational Database Management System (RDBMS) like MySQL or PostgreSQL. A relational database is critical because it enforces strict data rules and guarantees ACID (Atomicity, Consistency, Isolation, Durability) transaction properties. For example:

- **Atomicity (A):** If a user tries to place an order, the database must *both* confirm the bank payment settlement and modify the available warehouse stock as Deducted. If the payment gateway fails halfway through, the entire transaction is rolled back automatically so the stock remains available for other active buyers, and the customer's wallet balance is not debited.
 - **Consistency (C):** The system prevents invalid database states. A retail listing price cannot accidentally be saved as negative ($\text{Price} \geq 0$), and a shipment tracking log cannot be finalized for an order transaction that does not physically exist in the system backend.
-

2. CORE BUSINESS DOMAINS & FEATURES

Based on our analysis of retail marketplace workflows, we have divided the core application layout into four distinct functional modules:

2.1 Customer Management Module

This module handles consumer profile states, login credentials, address directories, and loyalty point tiers.

- **Business Rules:**
 - Every customer must provide a unique email address during account registration. This email acts as their primary login credential and cannot be shared across multiple profiles.
 - A shopper is assigned a unique CustomerID upon registration, alongside their Name, Mobile Number, and a system-generated Account Creation Date.

- Users can progress through a "Loyalty Status" tiering network (such as Bronze, Silver, Gold) which tracks their cumulative purchase history and issues specific discount point multipliers.
- A consumer record can exist inside the relational system without having placed any active purchase orders yet.

2.2 Product & Inventory Management Module

This module houses the database records for active merchant suppliers, item categorization trees, warehouse stock levels, and public pricing sheets.

- **Business Rules:**

- Products are organized by distinct "Categories." Every retail item must belong to at least one primary Category (e.g., "Electronics", "Apparel").
- Physical warehouses are divided into separate storage areas. Each category contains a localized layout of specific racks and bins for inventory handling.
- A Stock Unit represents a specific variant of an item held in a designated warehouse location (e.g., "Size M Red Shirt on Rack 3, Bin B").
- The database must enforce the constraint that an active warehouse bin cannot host overlapping items from completely conflicting hazardous categories ($\neg \text{Stored}_{\text{Category1}} \wedge \text{Stored}_{\text{Category2}}$).

2.3 Shopping Cart & Order Management Lifecycle

This module acts as the core transaction engine, capturing which item combinations are selected, what inventory holds are locked down, and how orders progress over time.

- **Business Rules:**

- A single purchase checkout event is identified by a unique OrderID. It is linked to exactly one Customer.
- An order can contain multiple different product selections (e.g., purchasing 3 distinct items simultaneously). This creates a many-to-many (M:N) relationship between Orders and Products, which is resolved using a bridging table called Order Line Items.
- **The Historic Price Rule:** The unit item cost recorded in Order Line Items must represent the price at the exact second of purchase. If the merchant decides to raise standard catalog retail prices next week, past purchase receipts must remain unchanged to preserve correct accounting.
- Orders must support a rigid lifecycle state machine: Cart Selected → Pending Payment → Confirmed → Shipped → Delivered.

2.4 Payment & Shipment Management Module

This section handles currency clearing verification, digital tracking code assignments, and courier distribution state logs.

- **Business Rules:**

- Each unique order has a single Payment profile. A Payment record cannot progress or exist without a corresponding valid OrderID (1:1 relationship).
- An order status flag cannot progress to Confirmed until its corresponding Payment status row is updated as Successful.

- Once confirmed, the system must generate a unique digital tracking barcode identifier for each shipping parcel. The shipment status updates to Delivered only when the mobile scanner used by the courier partner validates the parcel barcode at the doorstep.

3. ACTOR PROFILES & SYSTEM INTERACTIONS

To maintain system security, different users must have distinct database access privileges. We define four primary actors:

Actor	Read Privileges	Write/Edit Rules
Customer	Product Catalog Listings, Personal Profile History	Create Carts, Generate Orders, Write Reviews
Administrator	Warehouse Inventory Logs, Daily Gross Metrics	Edit Product Prices, Add New Categories
Delivery Partner	Delivery Status Keys, Basic Address Identifiers	Update Shipping Logs, Complete Deliveries
System Admin	Full Database Schema Tables, System Core Files	CRUD Database Privileges, Edit User Roles

3.1 Customer (Shopper)

- **What they can Read:** Public product descriptions, current sale prices, stock status values, their own personal account details, active shopping cart tables, and historic order logs.
- **What they can Write/Edit:** Their user registration field details, saved shipping destinations, product item amounts in their active cart, new checkout orders, and feedback reviews on items purchased.
- **Safety Boundary:** A Customer can never view another user's personal details, credit cards, or order histories. They cannot change product pricing or adjust warehouse inventory allocations.

3.2 Administrator (Store Manager)

- **What they can Read:** Warehouse inventory counts, platform sales statistics, customer support feedback logs, and total revenue lines.
- **What they can Write/Edit:** Schedule price changes, introduce new items to the store catalog, modify stock supply numbers, and flag a warehouse section as "Under Maintenance".
- **Safety Boundary:** Managers cannot view customer credit cards, manually override transactional payment gateways, or delete financial audit histories.

3.3 Delivery Partner (Logistics Agent)

- **What they can Read:** Active shipping addresses, customer names, telephone contact lines, and assigned package parcel barcodes.
- **What they can Write/Edit:** Update package tracking states from Shipped → Delivered upon completing a verified doorstep drop-off.
- **Safety Boundary:** They cannot view customer financial records, edit product catalog attributes, cancel order transactions, or alter pricing data.

3.4 System Admin (Database Administrator)

- **What they can Read/Write:** Complete CRUD (Create, Read, Update, Delete) privileges across all database tables. All Admin transactions are logged to an audit table to prevent internal fraud.
-

4. ANALYTICAL REPORTING REQUIREMENTS

To help management optimize retail performance, the database must be structurally capable of executing aggregate joins and calculations to produce these three business reports:

- **Report 1: Low-Stock Product Alerts**
 - **Business Value:** Identifies fast-selling products before they completely run out, allowing purchasing managers to run smart restock supply orders.
 - **Logical Query Goal:** Find all products in the active catalog where the current warehouse quantity is less than or equal to a critical warning threshold **$\text{StockLevel} \leq O_t$ (where $O_t = 15\% \text{ of Max Capacity}$)**. The report must list the Product Title, Category, Supplier Name, Total Maximum Capacity, and the count of missing restock units.
 - **Report 2: Consumer Loyalty Report**
 - **Business Value:** Allows the marketing team to distribute free promotional vouchers or premium tier upgrades to high-spending shoppers.
 - **Logical Query Goal:** Retrieve the top 10% of all registered shoppers ranked by their total successful transaction checkouts dollar volume over the past 12 months. The report must list the Customer's Name, Email Address, and total amount spent on goods.
 - **Report 3: Category Revenue Performance Directory**
 - **Business Value:** Shows which item categories generate the most net profit, guiding future warehouse inventory acquisitions.
 - **Logical Query Goal:** For a specified calendar month, calculate the total gross checkout revenue generated, the total count of item pieces sold, and the average customer feedback rating for each distinct Category.
-

5. NON-FUNCTIONAL REQUIREMENTS & DATABASE CONSTRAINTS

Our system rules ensure the database is secure, performant, and reliable under real-world conditions:

- **Strict Data Constraints:**
 - Review rating submissions must be integers between 1 and 5 inclusive ($1 \leq \text{Rating} \leq 5$).
 - Listed item prices must be strictly positive values ($\text{Price} > 0$).
 - The inventory tracking engine must enforce that a single physical item piece cannot be double-assigned to two separate customer checkouts for the exact same order execution timestamp.
- **Concurrency Controls:**
 - If two users click "Buy Laptop Unit A10" at the exact same millisecond for the final stock item the **TrendWave** relational engine must enforce **Row-Level Transaction Locks**. The first user's request will lock the specific row representing that product stock entry, process their credit validation, set the

inventory line to Sold, and release the lock. The second user's transaction will then read the updated state (Sold) and gracefully inform them that the item is no longer available.

- **Historical Auditing & Safe Deletion:**

- To comply with financial accounting and data privacy regulations, user accounts cannot be permanently hard-deleted. If a user deletes their account, their profile status is updated to Inactive, and personal identifiers (like mobile numbers and billing addresses) are blanked out for privacy compliance. However, their past purchase transaction numbers and invoice line records must remain in our database to ensure that financial audits and historical sales reports balance perfectly.
-